

Emular con QEMU

OdC 2025

Instalación

0-TENER ACTUALIZADOS LOS REPOSITORIOS

```
$ sudo apt update
```

1- INSTALAR EL TOOLCHAIN PARA AARCH64

```
$ sudo apt install gcc-aarch64-linux-gnu
```

2- INSTALAR QEMU PARA ARM (incluye aarch64)

```
$ sudo apt install qemu-system-arm
```

3- INSTALAR GDB PARA AARCH64

```
$ sudo apt install gdb-multiarch
```

4- CONFIGURAR GDB PARA UTILIZAR GDB-DASHBOARD

```
$ wget -P ~ git.io/.gdbinit
```

Toolchain: Conjunto de herramientas que sirven para compilar, linkear, etc. código fuente.
Generalmente se usan en cadena de forma que la salida de un proceso es la entrada del otro

QEMU: Emulador y virtualizador de computadoras de código abierto.

GDB: Debugger de código abierto. Nos permite ejecutar el código paso por paso y analizar el estado del procesador.

GDB Dashboard: Archivo de configuración para gdb que brinda una interfaz gráfica con la información relevante del procesador.

En Windows primero se debe instalar WSL (preferentemente ubuntu) y en esa terminal ejecutar los mismos comandos (más información en el tutorial del aula virtual)

Ejecución del QEMU+GDB

1 - Bajar el template desde el aula virtual:

<https://famaf.aulavirtual.unc.edu.ar/mod/resource/view.php?id=22726>

2 - Ensamblar el código:

```
$ make
```

3 - Abrir una terminal **en el directorio del template** y ejecutar:

```
$ make runQemu
```

Esto inicia el emulador, es igual que ejecutar el siguiente comando:

```
$ qemu-system-aarch64 -s -S -machine virt -cpu cortex-a53 -machine type=virt -nographic -smp 1  
-m 64 -kernel kernel.img
```

4 - En **otra** terminal (sin cerrar la otra), **en el mismo directorio**, ejecutar el comando:

```
$ make runGDB
```

Esto corre el debugger y se conecta al emulador. Es igual que ejecutar el comando:

```
$ gdb-multiarch -ex "set architecture aarch64" -ex "target remote localhost:1234"
```

Ejecución del QEMU+GDB

Terminal 2

Terminal 1

```
gvdanovic@gvdanovic$ make
aarch64-linux-gnu-as -g --fatal-warnings main.s -o main.o
aarch64-linux-gnu-ld main.o -T memmap -o main.elf -M > memory_map.txt
aarch64-linux-gnu-ld: warning: main.elf has a LOAD segment with RNX permissions
aarch64-linux-gnu-objdump -D main.elf > main.list
aarch64-linux-gnu-objcopy main.elf -O ihex main.hex
aarch64-linux-gnu-objcopy main.elf -O binary kernel.img
gvdanovic@gvdanovic$ make runQEMU
qemu-system-aarch64 -s -S -machine virt -cpu cortex-a53 -machine type=virt -nagra
phic -smp 1 -m 64 -kernel kernel.img
```

```
Terminal
Assembly
0x0000000040000000 ? ldr x0, 0x40000010
0x0000000040000004 ? mov x1, xzr
0x0000000040000008 ? mov x2, xzr
0x000000004000000c ? mov x3, xzr
0x0000000040000010 ? ldr x4, 0x40000020
0x0000000040000014 ? br x4
0x0000000040000018 ? .inst 0x42000000 ; undefined
0x000000004000001c ? udf #0
0x0000000040000020 ? .inst 0x40000000 ; undefined
0x0000000040000024 ? udf #0

Breakpoints
Expressions
History
Memory
Registers
x0 0x0000000000000000 x1 0x0000000000000000 x2 0x0000000000000000 x3 0x0000000000000000 x4 0x0000000000000000 x5 0x0000000000000000 x6 0x0000000000000000
x7 0x0000000000000000 x8 0x0000000000000000 x9 0x0000000000000000 x10 0x0000000000000000 x11 0x0000000000000000 x12 0x0000000000000000 x13 0x0000000000000000
x14 0x0000000000000000 x15 0x0000000000000000 x16 0x0000000000000000 x17 0x0000000000000000 x18 0x0000000000000000 x19 0x0000000000000000 x20 0x0000000000000000
x21 0x0000000000000000 x22 0x0000000000000000 x23 0x0000000000000000 x24 0x0000000000000000 x25 0x0000000000000000 x26 0x0000000000000000 x27 0x0000000000000000
x28 0x0000000000000000 x29 0x0000000000000000 x30 0x0000000000000000 sp 0x0000000000000000 pc 0x0000000040000000 cpsr 0x400003c5

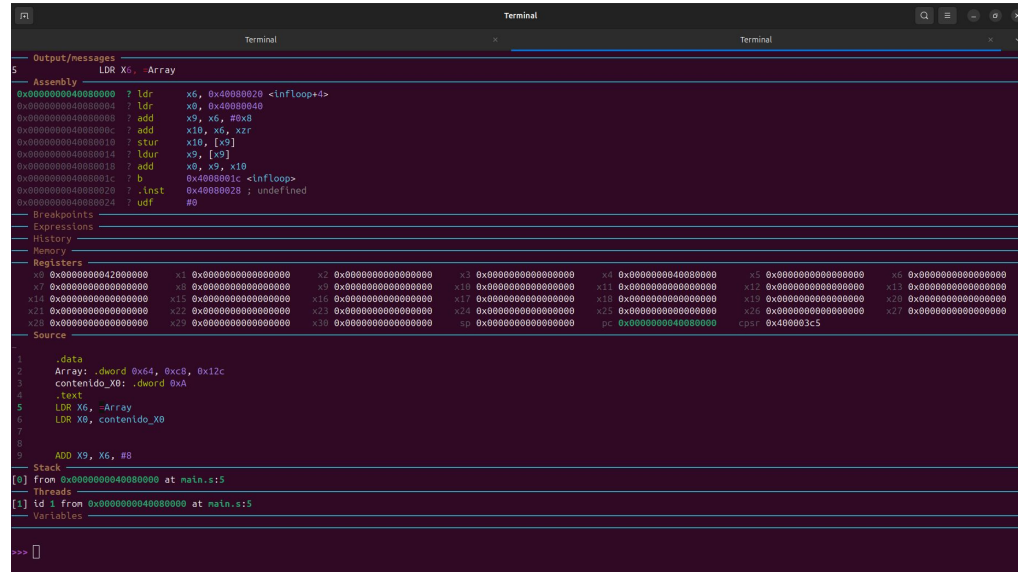
Source
Stack
(0) from 0x0000000040000000
Threads
(1) ld 1 from 0x0000000040000000
Variables
***
```

Agregar los símbolos

Agregar los símbolos permite poder ver el código escrito y poner breakpoints en forma más sencilla:

```
>>> add-symbol-file main.o 0x0000000040080000
```

Las primeras instrucciones no forman parte del código sino de la emulación, deben ser ejecutadas hasta que aparezca el código



```
Terminal
Output/messages
5 LDR X6, -Array
Assembly
0x0000000040080000 ? ldr x6, 0x40080028 <inloop+4>
0x0000000040080004 ? ldr x0, 0x40080040
0x0000000040080008 ? add x9, x6, #0x0
0x000000004008000c ? add x10, x6, xzr
0x0000000040080010 ? stur x10, [x9]
0x0000000040080014 ? ldur x9, [x0]
0x0000000040080018 ? add x0, x9, x10
0x000000004008001c ? b 0x4008001c <inloop>
0x0000000040080020 ? .lnst 0x40080028 ; undefined
0x0000000040080024 ? udf #0
Breakpoints
Expressions
History
Memory
Registers
x1 0x0000000042000000 x1 0x0000000000000000 x2 0x0000000000000000 x3 0x0000000000000000 x4 0x0000000040080000 x5 0x0000000000000000 x6 0x0000000000000000
x7 0x0000000000000000 x8 0x0000000000000000 x9 0x0000000000000000 x10 0x0000000000000000 x11 0x0000000000000000 x12 0x0000000000000000 x13 0x0000000000000000
x14 0x0000000000000000 x15 0x0000000000000000 x16 0x0000000000000000 x17 0x0000000000000000 x18 0x0000000000000000 x19 0x0000000000000000 x20 0x0000000000000000
x21 0x0000000000000000 x22 0x0000000000000000 x23 0x0000000000000000 x24 0x0000000000000000 x25 0x0000000000000000 x26 0x0000000000000000 x27 0x0000000000000000
x28 0x0000000000000000 x29 0x0000000000000000 x30 0x0000000000000000 x31 0x0000000000000000 pc 0x0000000040080000 cpsr 0x40083c5
Source
1 .data
2 Array: .dword 0x64, 0xc0, 0x12c
3 contenido_X0: .dword 0xA
4 .text
5 LDR X6, -Array
6 LDR X6, contenido_X0
7
8
9 ADD X9, X6, #8
Stack
[0] from 0x0000000040080000 at main.s:5
Threads
[1] Id 1 from 0x0000000040080000 at main.s:5
Variables
>>>
```

Cómo debuggear

El comando que se utiliza para ejecutar una única instrucción de assembly es:

```
>>> stepi
```

Si se desea ejecutar un bloque de n instrucciones:

```
>>> stepi n
```

Otra forma de ejecutar más de una instrucción es utilizando *breakpoints*:

```
>>> break n_instr
```

```
>>> break etiqueta
```

Luego, el comando *continue* se ejecuta el programa hasta encontrar el primer *breakpoint*.

```
>>> continue
```

Para ver todos los *breakpoints* existentes, se utiliza el comando:

```
>>> info breakpoints
```

Para eliminar todos *breakpoints* se utiliza el comando *delete*:

```
>>> delete breakpoints
```

Otros comandos útiles de GDB

Mostrar un segmento de memoria:

Considerando que *dir* es la dirección base de la memoria a partir de donde se desea ver el contenido y *n_byte* la cantidad de bytes que se mostrarán a partir de dicha dirección, se debe utilizar el siguiente comando:

```
>>> dashboard memory watch dir n_byte
```

Ejemplo:

```
>>> dashboard memory watch 0x0000000040080000 128
```

- Eliminar del dashboard los segmentos de memoria que se están mostrando:

```
>>> dashboard memory clear
```

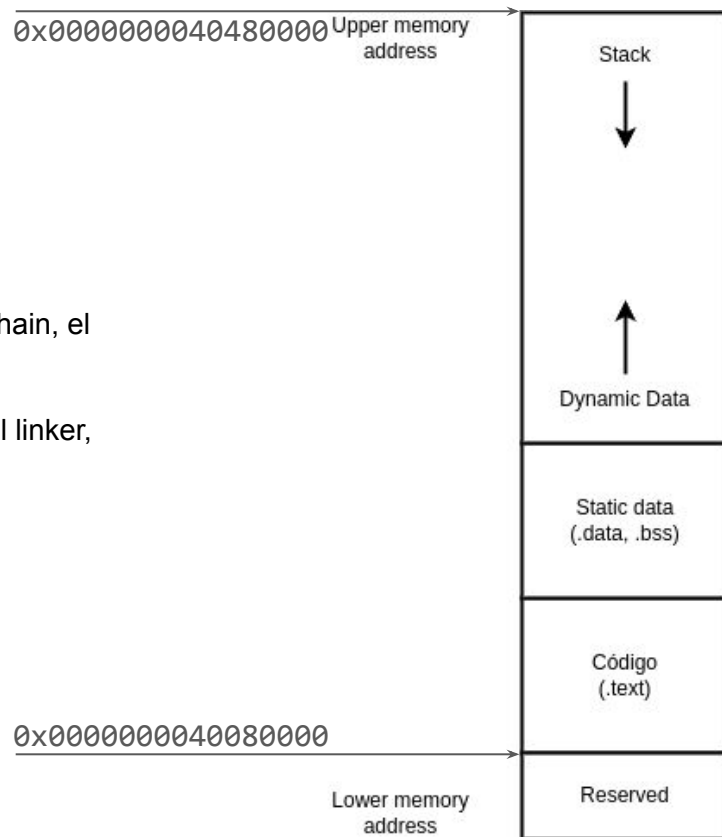
- Mostrar en la sección “Assembly” el opcode de una instrucción que se está por ejecutar:

```
>>> dashboard assembly -style opcodes 1
```

Archivos del template

El template cuenta con tres archivos:

- **main.s**: Archivo donde se escribe el código a emular
- **makefile**: Archivo donde se llaman las distintas herramientas del toolchain, el emulador y el debugger
- **memmap**: En este archivo se definen las secciones necesarias para el linker, la dirección de inicio y el tamaño de la ram.



Código

```
.data
    Array: .dword 0x64, 0xc8, 0x12c
    X_const: .dword 0xA
.text
    LDR X6, =Array      # Guardar en X6 la dirección del array
    LDR X0, X_const      # Guardar en X0 el contenido de X_const

    ADD X9, X6, #8
    ADD X10, X6, XZR
    STUR X10, [X9, #0]
    LDUR X9, [X9, #0]
    ADD X0, X9, X10

infloop: B infloop
```

Inicializar memoria

Código

Loop infinito

Sintaxis

Inicializar memoria:

[Etiqueta]: .[Tipo de dato] [dato1], [dato2], [dato3], [dato4]

```
.data
    Array: .dword 0x64, 0xc8, 0x12c
    X_const: .dword 0xA
```

Traer datos de memoria a registros:

Se usa la instrucción LDR (load register)

El símbolo = implica que lo que se carga es la dirección.

```
.text
    LDR X6, =Array
    LDR X0, X_const
```

De C a Assembly

La toolchain cuenta con un compilador (gcc) que convierte código en C a **ARMv8**

El compilador se ejecuta con el comando **aarch64-linux-gnu-gcc**, ej:

```
echo "long foo(long *A, long *B, long i, long j) {B[8] = A[i - j];}" | aarch64-linux-gnu-gcc -O1 -S -o- -xc -
```

```
.arch armv8-a
.file    "<stdin>"
.text
.align 2
.global foo
.type    foo, %function
foo:
.LFB0:
.cfi_startproc
sub     x2, x2, x3
ldr     x0, [x0, x2, lsl 3]
str     x0, [x1, 64]
ret
.cfi_endproc
.LFE0:
.size   foo, .-foo
.ident  "GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0"
.section .note.GNU-stack,"",@progbits
```



Alerta de código ARMv8!!!

Memoria en QEMU

```
>>> dashboard memory watch 0x0000000040080000 128
```

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Memory																
0x0000000040080000																
0x0000000040080000	06	01	00	58	e0	01	00	58	c9	20	00	91	ca	00	1f	8b
0x0000000040080010	2a	01	00	f8	29	01	40	f8	20	01	0a	8b	00	00	00	14
0x0000000040080020	28	00	08	40	00	00	00	00	64	00	00	00	00	00	00	00
0x0000000040080030	c8	00	00	00	00	00	00	00	2c	01	00	00	00	00	00	00
0x0000000040080040	0a	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0000000040080050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0000000040080060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0000000040080070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

The diagram shows two horizontal curly braces at the bottom of the memory table. The left brace, labeled 'Dirección' in blue, spans the first column containing memory addresses. The right brace, labeled 'Contenido' in green, spans the remaining 16 columns containing the hexadecimal values of the memory content.